

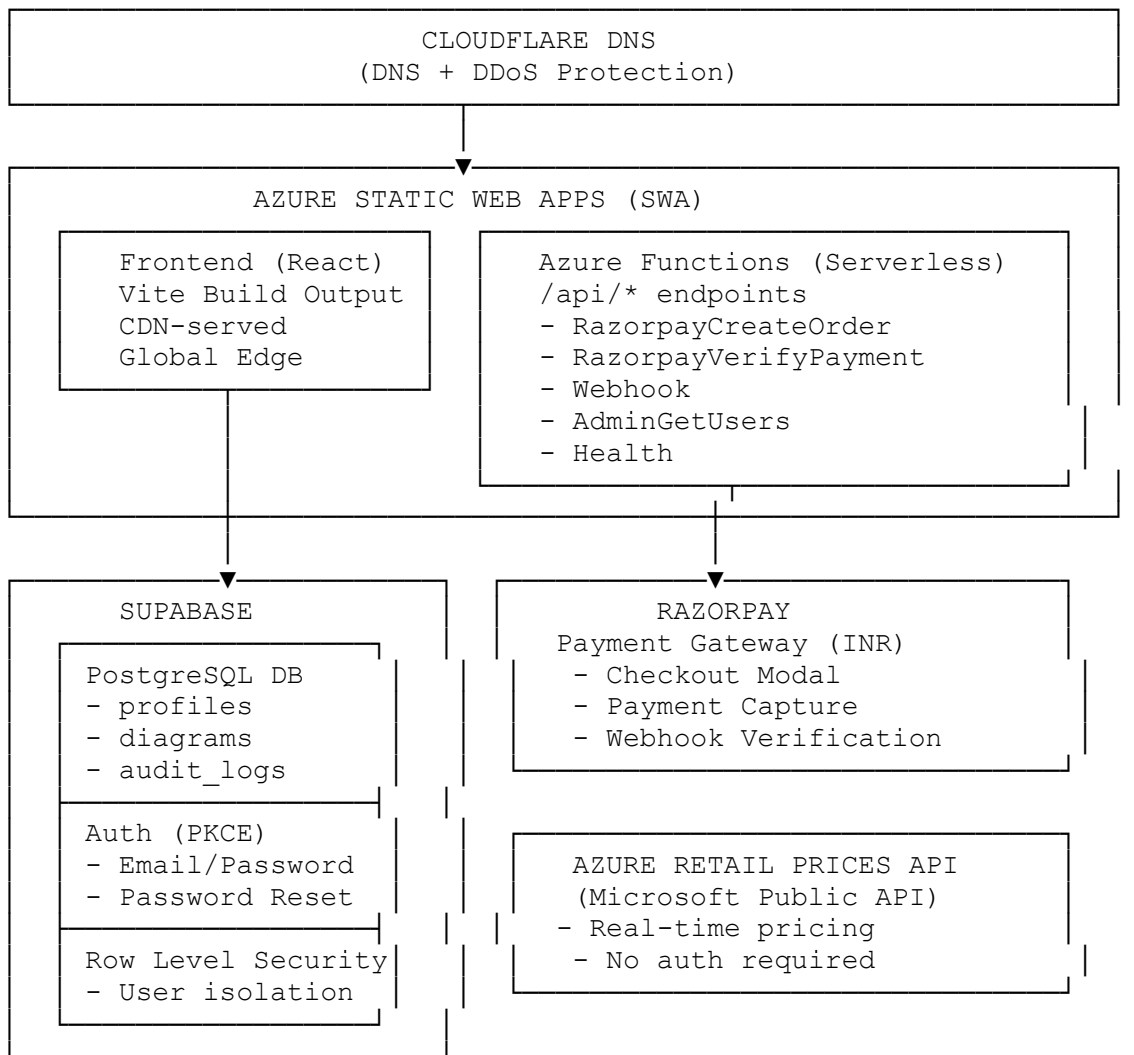
Cloud Canvas Designer – Architecture Assessment & Technical SOP

Version: 1.0.0
Date: April 2, 2026
Author: Arunim's IT Café
Classification: Internal / Technical

1. Architecture Overview

1.1 System Diagram

...



...

1.2 Component Inventory

Component	Service	Tier	Purpose
-----	-----	-----	-----

Frontend	Azure Static Web Apps	Free	React SPA hosting, global CDN
Backend API	Azure Functions (managed by SWA)	Free	Payment processing, admin APIs
Database	Supabase (PostgreSQL)	Free	User profiles, diagrams, audit logs
Authentication	Supabase Auth	Free	Email/password, PKCE flow, password reset
Payments	Razorpay	Live	INR payment processing
DNS	Cloudflare	Free	DNS resolution, DDoS protection
Pricing API	Azure Retail Prices API	Free	Real-time Azure service pricing
CI/CD	GitHub Actions	Free	Auto-deploy on push to main
Icons	Microsoft Azure Icon Library	Free	700+ official Azure service SVGs

1.3 Cost Structure

Service	Monthly Cost
Azure SWA (Free tier)	\$0
Supabase (Free tier)	\$0
Cloudflare DNS (Free)	\$0
GitHub Actions	\$0 (2000 min/month free)
Azure Retail Prices API	\$0 (public, no auth)
Razorpay	2% per transaction
Total Infrastructure	**\$0/month**

2. Backend Architecture

2.1 How the Backend Works

Cloud Canvas Designer runs as a ****serverless JAMstack**** application. There is no traditional backend server.

****Frontend (Client-Side):****

- React 19 SPA built with Vite 7
- Served via Azure SWA's global CDN
- All diagram logic (canvas, validation, export) runs in the browser
- Communicates directly with Supabase for auth and data

****Azure Functions (Serverless API):****

- Located in ``/api/`` directory
- Automatically deployed with SWA
- Only used for: payment order creation, payment verification, admin user queries, RLS fixes
- Cold start: ~1-2 seconds, warm: <100ms

****Supabase (Database + Auth):****

- PostgreSQL database with 3 tables: ``profiles``, ``diagrams``, ``audit_logs``
- Row Level Security (RLS) ensures users can only access their own data
- Admin users bypass RLS via ``is_admin()`` SQL function

- Auth uses PKCE flow with email/password (no OAuth providers configured)

2.2 Data Flow

...

User Action	→ Where it runs	→ Data stored
Sign up	→ Supabase Auth	→ auth.users + profiles
Login	→ Supabase Auth	→ Session (localStorage)
Draw diagram	→ Browser (React state)	→ In-memory only
Save to cloud	→ Supabase REST API	→ diagrams table
Export TF/Bicep/ARM	→ Browser (JS)	→ Downloaded file
Export PNG/PDF	→ Browser (html2canvas)	→ Downloaded file
Payment	→ Razorpay Checkout	→ Razorpay +
profiles.subscription_tier		
Cost estimation	→ Azure Retail API	→ Cached in-memory (1hr)
WAF Validation	→ Browser (JS)	→ Displayed in modal
Audit logging	→ Supabase REST API	→ audit_logs table

...

2.3 Database Schema

****profiles**** (user data):

```sql

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| id                      | UUID (PK, references auth.users)                   |
| email                   | TEXT                                               |
| name                    | TEXT                                               |
| role                    | TEXT (architect   admin)                           |
| subscription_tier       | TEXT (trial   starter   professional   enterprise) |
| trial_start_date        | TIMESTAMPTZ                                        |
| trial_expires_at        | TIMESTAMPTZ                                        |
| trial_exports_used      | INTEGER                                            |
| diagrams_created        | INTEGER                                            |
| is_active               | BOOLEAN                                            |
| upgraded_at             | TIMESTAMPTZ                                        |
| subscription_expires_at | TIMESTAMPTZ                                        |
| created_at              | TIMESTAMPTZ                                        |

```

****diagrams**** (cloud-saved diagrams):

```sql

|                  |                                        |
|------------------|----------------------------------------|
| id               | UUID (PK)                              |
| user_id          | UUID (FK → auth.users)                 |
| name             | TEXT                                   |
| share_id         | TEXT (UNIQUE, for public sharing)      |
| data             | JSONB (items, connections, boundaries) |
| item_count       | INTEGER                                |
| connection_count | INTEGER                                |
| created_at       | TIMESTAMPTZ                            |
| updated_at       | TIMESTAMPTZ                            |

```

****audit_logs**** (security events):

```sql

|            |                                         |
|------------|-----------------------------------------|
| id         | UUID (PK)                               |
| user_id    | UUID                                    |
| email      | TEXT                                    |
| event      | TEXT (LOGIN, SIGNUP, LOGOUT, etc.)      |
| details    | JSONB (device info, IP, action details) |
| ip_address | TEXT                                    |
| created_at | TIMESTAMPTZ                             |

---

## ## 3. Security Assessment

### ### 3.1 Authentication

- Supabase Auth with PKCE flow (no implicit grant)
- Passwords hashed by Supabase (bcrypt)
- Session stored in localStorage with auto-refresh
- Password reset via Supabase email link
- Admin emails centralized in `adminConfig.js` + env var override

### ### 3.2 Authorization

- Row Level Security (RLS) on all Supabase tables
- Users can only read/write their own profiles and diagrams
- Admin access via `is\_admin()` SQL function (no recursion)
- Protected routes in React (`ProtectedRoute` component)
- Admin dashboard restricted to admin role

### ### 3.3 Payment Security

- Razorpay handles all card data (PCI DSS compliant)
- No card details stored on our servers
- Payment captured client-side via Razorpay handler
- Subscription tier updated in Supabase after payment
- \*\*Gap:\*\* No server-side payment signature verification (webhook stub only)

### ### 3.4 Data Security

- All Supabase communication over HTTPS
- Anon key is public (safe - RLS protects data)
- Service role key only on server-side (Azure Functions env vars)
- `.env` file gitignored
- GitHub Secrets for production credentials

### ### 3.5 Known Security Gaps

| Gap                                           | Risk   | Mitigation                             |
|-----------------------------------------------|--------|----------------------------------------|
| No server-side payment verification           | Medium | Razorpay dashboard manual verification |
| Client-side IP detection (ipify.org)          | Low    | Best-effort, non-blocking              |
| No rate limiting on Supabase                  | Low    | Supabase has built-in rate limits      |
| No CAPTCHA on signup                          | Low    | Add if abuse detected                  |
| Trial watermark easily removable via DevTools | Low    | Acceptable for current scale           |

---

## ## 4. Feature Matrix

| Feature                    | Trial | Professional (₹2,000/mo) | Enterprise (₹6,699/mo) |
|----------------------------|-------|--------------------------|------------------------|
| Drag & drop canvas         | ✓     | ✓                        | ✓                      |
| 700+ Azure icons           | ✓     | ✓                        | ✓                      |
| WAF validation (5 pillars) |       | ✓                        | ✓                      |
| Real-time cost estimation  |       | ✓                        | ✓                      |
| Templates (50)             | ✓     | ✓                        | ✓                      |
| Undo/Redo                  | ✓     | ✓                        |                        |
| Diagrams                   | 3 max | 10,000                   | Unlimited              |
| PNG export                 | 5 max | Unlimited                | Unlimited              |
| PDF export                 | ✗     | ✓                        | ✓                      |
| Terraform export           | ✗     | ✓                        | ✓                      |
| Bicep export               | ✗     | ✓                        | ✓                      |
| ARM template export        | ✗     | ✓                        | ✓                      |
| Cloud save (My Diagrams)   |       | ✓                        | ✓                      |
| Share links                | ✓     | ✓                        | ✓                      |
| Version history            | ✓     | ✓                        | ✓                      |
| Region cost comparison     |       | ✓                        | ✓                      |
| Presentation mode          | ✓     | ✓                        | ✓                      |
| Terraform paste/import     |       | ✗                        | ✓                      |
| Priority support           | ✗     | ✓                        | ✓                      |
| SSO/SAML                   | ✗     | ✗                        | ✓                      |
| API access                 | ✗     | ✗                        | ✓                      |
| SLA guarantee              | ✗     | ✗                        | ✓                      |

---

## ## 5. Technical SOP (Standard Operating Procedures)

### ### 5.1 Deployment

```
```bash
# Code is auto-deployed via GitHub Actions on push to main
git add -A
git commit -m "description"
git push origin main
# GitHub Actions → npm install → npm run build → deploy to Azure SWA
# Takes ~2-3 minutes
```
```

### ### 5.2 Environment Variables

```
```
**Local (.env):**
```
```

```
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJ...
VITE_RAZORPAY_KEY_ID=rzp_live_xxx
```

```
VITE_ADMIN_EMAILS=admin@example.com
```
```

```
**Production (GitHub Secrets):**
```

```
- `AZURE_STATIC_WEB_APPS_API_TOKEN_PURPLE_PEBBLE_081EF0400`
- `VITE_SUPABASE_URL`
- `VITE_SUPABASE_ANON_KEY`
- `VITE_RAZORPAY_KEY_ID`
```

```
### 5.3 Adding a New Admin User
```

1. Add email to `VITE\_ADMIN\_EMAILS` env var (comma-separated)
2. Or add to `src/utils/adminConfig.js` DEFAULT_ADMIN_EMAILS array
3. Redeploy

```
### 5.4 Extending a User's Trial
```

1. Login to admin dashboard (`/admin`)
2. Find user → click "+  Trial"
3. Enter number of days → confirm

```
### 5.5 Changing a User's Plan
```

1. Admin dashboard → find user → click "  Plan"
2. Select new plan → confirm
3. If downgrading to trial: fresh 7-day trial is granted automatically

```
### 5.6 Database Maintenance
```

```
```sql
```

```
-- Check user count by plan
```

```
SELECT subscription_tier, COUNT(*) FROM profiles GROUP BY
subscription_tier;
```

```
-- Find expired trials
```

```
SELECT email, trial_expires_at FROM profiles
WHERE subscription_tier = 'trial' AND trial_expires_at < NOW();
```

```
-- Check audit log volume
```

```
SELECT event, COUNT(*) FROM audit_logs GROUP BY event ORDER BY count
DESC;
```

```
```
```

```
### 5.7 Monitoring
```

- Audit logs: Admin dashboard → Audit Logs tab
- User metrics: Admin dashboard → stats cards (total, trial, paid, banned)
- Supabase dashboard: Database size, API requests, auth events
- GitHub Actions: Deployment status and build logs

```
---
```

```
## 6. FAQ
```

```
### General
```

```
**Q: Where is the app hosted?**
```

A: Azure Static Web Apps (Free tier) with Cloudflare DNS in front. Frontend is CDN-served globally.

****Q: Is there a backend server?****

A: No traditional server. Azure Functions (serverless) handle payment APIs. All other logic runs client-side or via Supabase REST API.

****Q: What happens if Supabase goes down?****

A: Auth and cloud save will fail. The canvas, templates, local save/load, and all export features continue to work (they're client-side only).

****Q: What happens if Razorpay goes down?****

A: Payment flow fails. All other features work. Users can retry payment later.

****Q: How much does it cost to run?****

A: \$0/month on free tiers. Razorpay charges 2% per transaction. Scale triggers: Supabase free tier limit (500MB DB, 50K auth users), SWA free tier (100GB bandwidth/month).

Technical

****Q: How are diagrams stored?****

A: Two ways – (1) Local JSON file download (Save button), (2) Cloud save to Supabase `diagrams` table as JSONB.

****Q: How does the Terraform parser work?****

A: Regex-based HCL parser. Extracts `resource` blocks, maps `azurerm\_\*` types to Azure icon IDs (136 types mapped), detects cross-resource references for connections, groups by resource_group_name for boundary generation.

****Q: How does cost estimation work?****

A: Primary: Azure Retail Prices API (live, cached 1hr). Fallback: static price estimates based on standard SKUs. Supports 20+ Azure regions and 5 currencies.

****Q: How does WAF validation work?****

A: Client-side rule engine scores architecture against 5 Azure Well-Architected Framework pillars. Checks for: single points of failure, missing security services, cost optimization opportunities, monitoring gaps, and performance bottlenecks.

****Q: Can users collaborate on diagrams?****

A: Not in real-time. Users can share read-only links via share IDs. Cloud-saved diagrams are per-user.

****Q: How are exports generated?****

A: All exports (Terraform, Bicep, ARM, PNG, PDF, JSON) are generated client-side in the browser. No server processing needed. Terraform exports as a ZIP with 5 files.

Security

****Q: Are passwords stored in the app?****

A: No. Supabase Auth handles all password hashing (bcrypt) and storage. The app never sees raw passwords.

****Q: Can users see other users' data?****

A: No. Supabase RLS policies ensure users can only access their own profiles and diagrams. Admin users can view all profiles via the admin dashboard.

****Q: Is payment data secure?****

A: Yes. Razorpay handles all card data (PCI DSS Level 1 compliant). No card details touch our servers or database.

****Q: What data is logged?****

A: Auth events (login, signup, logout, password reset), admin actions (plan changes, bans), and session heartbeats. Each log includes: timestamp, event type, email, IP address, device info (OS, browser, screen, timezone).

Troubleshooting

****Q: Admin dashboard shows only 1 user (RLS blocking)****

A: Run the RLS fix SQL in Supabase SQL Editor (shown in the admin dashboard warning banner), or set up the backend Azure Function with `SUPABASE\_SERVICE\_ROLE\_KEY`.

****Q: Password reset email goes to localhost****

A: Set the Site URL in Supabase Dashboard → Authentication → URL Configuration to your production URL.

****Q: Deployment fails on GitHub Actions****

A: Check that all 4 GitHub Secrets are set:
`AZURE\_STATIC\_WEB\_APPS\_API\_TOKEN\_\*`, `VITE\_SUPABASE\_URL`,
`VITE\_SUPABASE\_ANON\_KEY`, `VITE\_RAZORPAY\_KEY\_ID`.

****Q: Cost panel shows \$0 for a service****

A: The service may not be mapped in the Azure Retail Prices API mapper. Enable "Live" pricing mode. If still \$0, the service uses indirect billing or isn't available in the selected region.

****Q: Terraform import shows items outside boundaries****

A: The TF code must use `resource\_group\_name = azurerm\_resource\_group.X.name` references for items to be grouped inside RG boundaries. Hardcoded string values won't be detected.

7. Technology Stack Summary

| Layer | Technology | Version |
|------------|--------------|---------|
| Framework | React | 19.2.0 |
| Build Tool | Vite | 7.3.1 |
| Routing | React Router | 7.13.1 |

| | | |
|------------|-------------------------|--------|
| Database | Supabase (PostgreSQL) | Latest |
| Auth | Supabase Auth (PKCE) | Latest |
| Payments | Razorpay | v2.9.6 |
| PDF Export | jsPDF + jspdf-autotable | 2.5.2 |
| PNG Export | html2canvas | 1.4.1 |
| ZIP Export | JSZip | 3.10.1 |
| Hosting | Azure Static Web Apps | Free |
| CI/CD | GitHub Actions | v4 |
| DNS | Cloudflare | Free |
| Linting | ESLint | 9.39.1 |

Document generated for Cloud Canvas Designer by Arunim's IT Café
Last updated: April 2, 2026